
Rapport Technique

Projet de Compilation

Compilateur pour le Langage TPC

De l'analyse syntaxique à la génération de code NASM

Vincent PLESSY

Licence d'Informatique – Année 2025-2026

Mars 2026

Université – Licence Informatique
Projet de Compilation – Semestre 6
Langage : TPC (sous-ensemble du C)
Cible : NASM ELF64 (Linux AMD64)

Table des matières

1	Introduction	3
2	Spécifications Techniques	3
2.1	Environnement de développement	3
2.2	Structure du projet	3
2.3	Métriques du code source	4
2.4	Codes de retour	4
3	Analyse Lexicale	4
3.1	Tokens reconnus	4
3.2	Gestion des erreurs lexicales	5
4	Analyse Syntaxique	5
4.1	Grammaire TPC	5
4.2	Priorités des opérateurs	6
5	Arbre de Syntaxe Abstraite (ASA)	6
5.1	Structure des nœuds	6
5.2	Labels de l'ASA	7
6	Tables des Symboles	7
6.1	Architecture	7
6.2	Structure d'un symbole	7
6.3	Allocation des variables locales	8
6.4	Fonctions prédéfinies	8
7	Analyse Sémantique	9
7.1	Vérifications effectuées	9
7.2	Algorithme général	9
7.3	Système de types	9
8	Génération de Code NASM	9
8.1	Architecture générale	10
8.2	Convention d'appel AMD64	10
8.3	Prologue et épilogue de fonction	10
8.4	Génération des expressions	11
8.5	Génération des instructions	11
8.6	Fonctions d'E/S prédéfinies	12
9	Extension : Structures	12
9.1	Déclaration et utilisation	12
9.2	Représentation en mémoire	13
10	Tests et Validation	13
10.1	Organisation des tests	13
10.2	Détail des tests	13
10.3	Résultats	14
10.4	Script de tests	14

11 Difficultés Rencontrées	15
11.1 Alignement de la pile AMD64	15
11.2 Gestion des codes de retour dans les tests	15
11.3 Tokens CHARACTER et séquences d'échappement	15
11.4 Variables locales vs paramètres	15
12 Résultats et Performances	15
12.1 Bilan fonctionnel	16
12.2 Limitations connues	16
13 Conclusion	16
A Annexes	16
A.1 Exemple de programme TPC compilé	16
A.2 Assembleur NASM généré (extrait)	17
A.3 Compilation et utilisation	17

1 Introduction

Ce rapport documente la réalisation du compilateur `tpcc` pour le langage TPC (*Tiny Procedural C*), dans le cadre du projet de Compilation de Licence 3.

Ce projet est la continuation directe du projet d'Analyse Syntaxique (Semestre 5), qui avait produit l'analyseur `tpcas` capable de détecter les erreurs lexicales et syntaxiques et de construire un Arbre de Syntaxe Abstraite (ASA). Le présent compilateur étend ce travail avec :

- une **analyse sémantique** complète (tables des symboles, vérification des types, contrôle des déclarations) ;
- une **génération de code** vers l'assembleur NASM (format ELF64, architecture AMD64 Linux) ;
- une suite de tests automatisée couvrant les quatre catégories attendues.

L'ensemble du code source peut être compilé avec un simple `make`.

2 Spécifications Techniques

2.1 Environnement de développement

Composant	Version / Description
Système d'exploitation	Linux AMD64 (Debian/Ubuntu)
Compilateur C	GCC 11+
Analyseur lexical	Flex 2.6+
Générateur de parseur	Bison 3.8+
Assembleur cible	NASM 2.15+ (format ELF64)
Éditeur de liens	ld (GNU Binutils)
Outil de construction	GNU Make

TABLE 1 – Outils utilisés

2.2 Structure du projet

```
project/
  makefile
  run_tests.sh      # Script de tests automatiques
  bin/
    tpcc            # Binaire compilé
  src/
    tpcc.lex        # Analyseur lexical (Flex)
    tpcc.y          # Parseur + point d'entrée (Bison)
    tree.h / tree.c # Module ASA
    symtable.h / symtable.c # Tables des symboles
    semantic.h / semantic.c # Analyse sémantique
    codegen.h / codegen.c  # Génération de code NASM
  test/
    good/           # 12 programmes corrects
```

```

syn-err/      # 5 erreurs lexicales/syntaxiques
sem-err/      # 10 erreurs sémantiques
warn/        # 3 programmes avec avertissements
rep/
  RAPPORT.tex    # Ce document

```

2.3 Métriques du code source

Fichier	Lignes	Rôle
tpcc.lex	~110	Lexer Flex
tpcc.y	~250	Parser Bison + main()
tree.c	~120	Module ASA
symtable.c	~220	Tables des symboles
semantic.c	~380	Analyse sémantique
codegen.c	~580	Génération NASM
Total	~1660	

TABLE 2 – Taille des fichiers sources

2.4 Codes de retour

Code	Signification
0	Compilation réussie (fichier <code>.asm</code> produit)
1	Erreur lexicale ou syntaxique
2	Erreur sémantique
3	Erreur interne / fichier introuvable

TABLE 3 – Codes de retour de `tpcc`

3 Analyse Lexicale

3.1 Tokens reconnus

L'analyseur lexical `tpcc.lex` reconnaît les catégories suivantes :

Catégorie	Description
TYPE	int, char, void
STRUCT	Mot-clé struct
IF / ELSE	Mots-clés de condition
WHILE	Boucle
RETURN	Instruction de retour
IDENT	Identificateurs ([a-zA-Z_] [a-zA-Z0-9_]*)
NUM	Entiers ([0-9]+)
CHARACTER	Littéral caractère ('x', '\n', etc.)
ADDSUB	+, -
DIVSTAR	*, /, %
EQ	==, !=
ORDER	<, <=, >, >=
AND / OR	&&,
NOT	!
Ponctuations	() { } [] ; , .

TABLE 4 – Tokens du lexer

3.2 Gestion des erreurs lexicales

Tout caractère non reconnu provoque l'émission d'un message d'erreur sur `stderr` et la valeur `lexical_error` est positionnée afin de retourner le code 1 en fin de parsing.

4 Analyse Syntaxique

4.1 Grammaire TPC

Le parseur `tpcc.y` implémente la grammaire complète du langage TPC. Les productions principales sont :

```

1 Prog      : DeclList FunctList
2            | FunctList
3            ;
4
5 DeclList   : DeclList Decl
6            | Decl
7            ;
8
9 Decl       : TYPE DeclVars ';'
10           | TYPE IDENT '[' NUM ']' ';'
11           | StructDef ';'
12           ;
13
14 FunctList  : FunctList Funct
15           | Funct
16           ;
17
18 Funct      : TypeFunct IDENT '(' Params ')' Body
19           ;

```

```

20
21 Instr      : Assign ';'
22             | StructAff ';'
23             | If
24             | While
25             | Return ';'
26             | FunctCall ';'
27             | '{' Body '}'
28             | ';'
29             ;

```

Listing 1 – Extrait de la grammaire Bison

4.2 Priorités des opérateurs

Les priorités sont déclarées via les directives Bison `%left` / `%right`, du plus faible au plus fort :

1. `||` (ou logique)
2. `&&` (et logique)
3. `==`, `!=` (égalité)
4. `<`, `<=`, `>`, `>=` (ordre)
5. `+`, `-` (addition/soustraction)
6. `*`, `/`, `%` (multiplication/division)
7. `!`, unaire `-` (opérateurs unaires)

5 Arbre de Syntaxe Abstraite (ASA)

5.1 Structure des nœuds

Le module `tree.h` / `tree.c` est identique à celui du projet S1. Chaque nœud de l'ASA est représenté par la structure :

```

1 typedef struct Node {
2     label_t      label;          /* type de nœud */
3     struct Node *firstChild;
4     struct Node *nextSibling;
5     int          lineno;
6     char         ident[MAXNAME]; /* pour Ident, Character */
7     int          num;            /* pour Num */
8     int          type;           /* type TPC (TY_INT, etc.) */
9 } Node;

```

Listing 2 – Structure d'un nœud ASA

5.2 Labels de l'ASA

Label	Description
Prog	Racine du programme
DeclVars	Liste de déclarations de variables
Declarator	Un déclarant (nom + éventuel tableau)
Funct	Définition de fonction
Params	Liste de paramètres formels
Param	Un paramètre
Body	Corps de fonction (liste d'instructions)
InstrAssign	Affectation
InstrIf	Condition simple
InstrIfElse	Condition avec alternative
InstrWhile	Boucle <code>while</code>
InstrCall	Appel de procédure
InstrReturn	<code>return expr</code>
InstrReturnVoid	<code>return</code> sans valeur
InstrBlock	Bloc <code>{...}</code>
F	Appel de fonction (dans expression)
Ident	Identificateur
Num	Entier littéral
Character	Caractère littéral
OpAddsub	Opérateur <code>+</code> / <code>-</code>
OpDivstar	Opérateur <code>*</code> / <code>/</code> / <code>%</code>
OpEq	Opérateur <code>==</code> / <code>!=</code>
OpOrder	Opérateurs de comparaison
OpAnd / OpOr	Opérateurs logiques
OpNot	Négation logique
OpUnaryMinus	Moins unaire
StructDef / StructVar / StructAff	Structures

TABLE 5 – Principaux labels de l'ASA

6 Tables des Symboles

6.1 Architecture

Le module `syntable.h` / `syntable.c` gère quatre tables globales :

- `g_structs` Définitions de structures (`struct`).
- `g_vars` Variables globales.
- `g_funcs` Fonctions déclarées (incluant les fonctions prédéfinies).
- `l_vars` Variables locales et paramètres de la fonction courante.

6.2 Structure d'un symbole

```

1 typedef struct Symbol {
2     char      name[MAX_NAME];

```

```

3   SymKind  kind;          /* SYM_GVAR, SYM_LVAR, SYM_PARAM,
                            SYM_FUNC, SYM_SDEF */
4   TpcType  type;          /* {kind:
                            TY_INT/TY_CHAR/TY_VOID/TY_STRUCT, struct_name} */
5   int      offset;        /* offset rbp pour locaux/params
                            (n gatif) */
6   int      size;          /* taille en octets */
7   int      nparams;       /* nb de param tres (pour les
                            fonctions) */
8   Symbol   *params;       /* liste cha n e des param tres */
9   Field    fields[MAX_FIELDS];
10  int      nfields;       /* nb de champs (pour les structs) */
11  int      struct_size;   /* taille totale de la struct */
12  struct Symbol *next;
13 } Symbol;

```

Listing 3 – Structure Symbol

6.3 Allocation des variables locales

Chaque variable locale ou paramètre occupe **8 octets** sur la pile (indépendamment de son type effectif), ce qui simplifie le calcul des offsets. L’offset de la première variable est -8 par rapport à `rbp`, puis -16 , -24 , etc.

```

1 void sym_add_lvar(const char *name, TpcType type) {
2   Symbol *s = calloc(1, sizeof(*s));
3   strncpy(s->name, name, MAX_NAME-1);
4   s->kind    = SYM_LVAR;
5   s->type    = type;
6   s->offset  = next_offset; /* n gatif, commence -8 */
7   s->size    = 8;
8   next_offset -= 8;
9   /* insertion en t te de l_vars */
10  s->next = l_vars;
11  l_vars = s;
12 }

```

Listing 4 – Allocation d’une variable locale

6.4 Fonctions prédéfinies

Les quatre fonctions d’E/S sont pré-enregistrées dans `g_funcs` lors de l’initialisation :

Nom	Paramètre	Retour
<code>putchar</code>	<code>int c</code>	<code>void</code>
<code>putint</code>	<code>int n</code>	<code>void</code>
<code>getchar</code>	<i>aucun</i>	<code>int</code>
<code>getint</code>	<i>aucun</i>	<code>int</code>

TABLE 6 – Fonctions prédéfinies TPC

7 Analyse Sémantique

7.1 Vérifications effectuées

L'analyse sémantique (`semantic.c`) parcourt l'ASA après le parsing et effectue les vérifications suivantes :

Vérification	Code retour
Variable non déclarée	erreur (2)
Variable déclarée deux fois	erreur (2)
Fonction non déclarée	erreur (2)
Fonction déclarée deux fois	erreur (2)
Nombre d'arguments incorrect	erreur (2)
Utilisation d'une fonction <code>void</code> comme expression	erreur (2)
Absence de fonction <code>main</code>	erreur (2)
<code>main</code> avec mauvaise signature	erreur (2)
Redéclaration locale d'un paramètre	erreur (2)
Affectation <code>int</code> $\rightarrow$ <code>char</code>	avertissement
<code>return int</code> dans fonction <code>char</code>	avertissement

TABLE 7 – Vérifications sémantiques

7.2 Algorithme général

1. Pré-enregistrement des fonctions prédéfinies.
2. Parcours des définitions de structures (`StructDef`).
3. Enregistrement des variables globales (`DeclVars`).
4. Pour chaque fonction :
 - (a) Enregistrement dans `g_funcs`.
 - (b) Réinitialisation de `l_vars`.
 - (c) Enregistrement des paramètres formels.
 - (d) Parcours récursif du corps (`Body`).
5. Vérification de la présence de `main()`.

7.3 Système de types

La compatibilité de types suit les règles :

- `int` $\rightarrow$ `int` : compatible (OK)
- `char` $\rightarrow$ `char` : compatible (OK)
- `char` $\rightarrow$ `int` : compatible implicite (OK, pas d'avertissement)
- `int` $\rightarrow$ `char` : compatible avec avertissement (code 0, message sur `stderr`)
- Toute autre combinaison : incompatible (erreur, code 2)

8 Génération de Code NASM

8.1 Architecture générale

Le module `codegen.c` traduit l'ASA en assembleur NASM ELF64. Le fichier généré est structuré en trois sections :

`section .bss` Variables globales (réservation de 8 octets par variable).
`section .text` Code : `_start`, toutes les fonctions, puis les fonctions prédéfinies.

```

1 section .bss
2     _x:  resb 8           ; variable globale x
3
4 section .text
5     global _start
6     extern putchar, putint, getchar, getint ; si non g n r s
7
8 _start:
9     call main
10    mov  rdi, rax
11    mov  rax, 60
12    syscall
13
14 main:
15    push rbp
16    mov  rbp, rsp
17    sub  rsp, 16
18    ; corps de la fonction
19    xor  eax, eax
20    leave
21    ret

```

Listing 5 – Structure d'un fichier `.asm` généré

8.2 Convention d'appel AMD64

Le compilateur suit la convention *System V AMD64 ABI* :

Registre	Rôle
<code>rdi</code>	1 ^{er} argument entier
<code>rsi</code>	2 ^e argument entier
<code>rdx</code>	3 ^e argument entier
<code>rcx</code>	4 ^e argument entier
<code>r8</code>	5 ^e argument entier
<code>r9</code>	6 ^e argument entier
<code>rax</code>	Valeur de retour

TABLE 8 – Registres AMD64 utilisés

8.3 Prologue et épilogue de fonction

Chaque fonction suit le schéma :

```

1 nom_fonction:
2   push rbp
3   mov  rbp, rsp
4   sub  rsp, N          ; N = taille pile locale (align e 16
                        ; octets)
5   ; sauvegarde des param tres :
6   mov  [rbp-8], rdi
7   mov  [rbp-16], rsi
8   ; ...
9   ; corps
10  xor  eax, eax        ; valeur de retour par d faut (0)
11  leave
12  ret

```

Listing 6 – Prologue/épilogue standard

L'alignement de la pile respecte la contrainte AMD64 : après `push rbp`, `rsp` est décalé de 8. On alloue donc N tel que $N \equiv 8 \pmod{16}$ (garantit que `rsp` est multiple de 16 avant tout `call`).

8.4 Génération des expressions

La fonction `cg_expr()` place le résultat dans `eax` :

Nœud ASA	Code généré
Num n	<code>mov eax, n</code>
Character ' c '	<code>mov eax, ord(c)</code>
Ident x (local)	<code>mov eax, [rbp+offset]</code>
Ident x (global)	<code>mov eax, [_x]</code>
OpAddsub $+$	éval gauche, push, éval droite, pop rcx, add
OpDivstar $*$	éval gauche, push, éval droite, pop rcx, imul
OpDivstar $/$	idiv (quotient dans eax)
OpEq $==$	<code>cmp + sete al + movzx</code>
OpAnd	court-circuit : saute si gauche = 0
OpOr	court-circuit : saute si gauche $\neq$ 0
OpNot	<code>test eax,eax + sete al</code>
OpUnaryMinus	<code>neg eax</code>
F (appel)	<code>push args, call, (pop args si besoin)</code>

TABLE 9 – Génération des expressions

8.5 Génération des instructions

InstrAssign Évalue l'expression droite dans `eax`, stocke dans la variable gauche (`[rbp+offset]` ou `[_x]`).

InstrIf Évalue la condition, saute à `_L<n>_end` si faux.

InstrIfElse Saute à `_L<n>_else` si faux, puis à `_L<n>_end` en fin de branche vraie.

InstrWhile Étiquette `_L<n>_start`, teste, corps, saute à `start`, étiquette `_L<n>_end`.

InstrReturn Évalue l'expression dans `eax`, puis `leave` / `ret`.

InstrBlock Parcours récursif des instructions enfants.

8.6 Fonctions d'E/S prédéfinies

Les quatre fonctions d'E/S sont implémentées directement en assembleur via les appels système Linux (sans bibliothèque C) :

```

1 putchar:
2     push rbp
3     mov  rbp, rsp
4     sub  rsp, 16
5     mov  [rbp-8], rdi      ; sauvegarde argument
6     mov  byte [rbp-9], dil ; octet      afficher
7     lea  rsi, [rbp-9]     ; pointeur vers le caract re
8     mov  rax, 1           ; syscall write
9     mov  rdi, 1           ; fd = stdout
10    mov  rdx, 1           ; longueur = 1
11    syscall
12    leave
13    ret

```

Listing 7 – Implémentation de `putchar` en NASM

```

1 putint:
2     ; convertit l'entier en cha ne ASCII, puis syscall write
3     ; g re le signe n gatif, les chiffres 0-9
4     ; utilise une pile de chiffres sur la pile locale

```

Listing 8 – Implémentation de `putint` en NASM (extrait)

9 Extension : Structures

9.1 Déclaration et utilisation

Le langage TPC supporte les structures :

```

1 struct Point {
2     int x;
3     int y;
4 };
5
6 int main(void) {
7     struct Point p;
8     p.x = 10;
9     p.y = 20;
10    putint(p.x + p.y);
11    return 0;
12 }

```

Listing 9 – Exemple d'utilisation d'une structure

9.2 Représentation en mémoire

Les structures sont stockées dans la table `g_structs`. Chaque champ est représenté par un `Field` (nom + type + offset interne). Le compilateur supporte l'accès aux champs via le nœud `StructAff` de l'ASA.

10 Tests et Validation

10.1 Organisation des tests

La suite de tests comprend **30 fichiers** répartis en quatre catégories :

Répertoire	Catégorie	Nb fichiers
test/good/	Programmes corrects (code 0)	12
test/syn-err/	Erreurs lexicales/syntaxiques (code 1)	5
test/sem-err/	Erreurs sémantiques (code 2)	10
test/warn/	Programmes avec avertissements (code 0)	3
Total		30

TABLE 10 – Répartition des tests

10.2 Détail des tests

Fichier	Description
<i>good/ – programmes corrects</i>	
01_main_simple.tpc	Programme minimal (<code>main</code> retourne 0)
02_variables_int.tpc	Déclarations et affectations d'entiers
03_arithmetic.tpc	Opérations arithmétiques (+, -, *, /, %)
04_if_else.tpc	Conditionnelle avec branche <code>else</code>
05_while.tpc	Boucle <code>while</code>
06_functions.tpc	Définition et appel de fonctions
07_global_vars.tpc	Variables globales
08_putint.tpc	Utilisation de <code>putint</code>
09_char_vars.tpc	Variables de type <code>char</code>
10_struct.tpc	Déclaration et accès à une structure
11_recursion.tpc	Fonction récursive (factorielle)
12_logical_ops.tpc	Opérateurs logiques (&&, , !)
<i>syn-err/ – erreurs syntaxiques</i>	
e01_missing_semicolon.tpc	Point-virgule manquant
e02_missing_brace.tpc	Accolade fermante absente
e03_invalid_expr.tpc	Expression invalide
e04_bad_struct.tpc	Structure mal formée
e05_bad_return.tpc	Opérateur sans opérande droite
<i>sem-err/ – erreurs sémantiques</i>	
s01_undeclared_var.tpc	Variable non déclarée

Fichier	Description
s02_undeclared_func.tpc	Fonction non déclarée
s03_duplicate_global.tpc	Variable globale déclarée deux fois
s04_duplicate_func.tpc	Fonction déclarée deux fois
s05_wrong_args.tpc	Nombre d'arguments incorrect
s06_void_as_expr.tpc	Fonction void utilisée en expression
s07_no_main.tpc	Absence de main
s08_bad_main.tpc	main avec mauvaise signature
s09_type_error.tpc	Incompatibilité de type
s10_local_param_same_name.tpc	Redéclaration locale d'un paramètre
<i>warn/ – programmes avec avertissements</i>	
w01_int_to_char.tpc	Affectation <code>int</code> → <code>char</code>
w02_int_return_in_char_func.tpc	<code>return int</code> dans fonction <code>char</code>
w03_implicit_char_to_int.tpc	Conversion implicite <code>char</code> → <code>int</code> (OK)

TABLE 11: Détail des fichiers de test

10.3 Résultats

Catégorie	Total	PASS	FAIL	Taux
good	12	12	0	100%
syn-err	5	5	0	100%
sem-err	10	10	0	100%
warn	3	3	0	100%
Total	30	30	0	100%

TABLE 12 – Résultats des tests automatiques

10.4 Script de tests

Le script `run_tests.sh` lance automatiquement les 30 tests et affiche un rapport coloré. Il supporte l'option `-asm` pour assembler et lier les programmes corrects avec `nasm + ld`.

```
$ ./run_tests.sh
=====
Tests du compilateur tpcc
=====

--- Programmes corrects (code de retour attendu: 0) ---
PASS [good] 01_main_simple.tpc (code=0)
...

--- Résultats finaux ---
PASS   : 30 / 30
FAIL   : 0 / 30
Score  : 100%
```

11 Difficultés Rencontrées

11.1 Alignement de la pile AMD64

La convention AMD64 exige que `rsp` soit aligné sur 16 octets avant toute instruction `call`. Après `push rbp`, `rsp` a déjà perdu 8 octets d'alignement. La formule d'allocation utilisée est :

$$N = \begin{cases} total & \text{if } total \equiv 8 \pmod{16} \\ \left\lceil \frac{total + 15}{16} \right\rceil \times 16 - 8 & \text{sinon} \end{cases}$$

où `total` est la somme des espaces nécessaires pour toutes les variables locales et les paramètres.

11.2 Gestion des codes de retour dans les tests

Le script `run_tests.sh` ne doit pas utiliser `set -e` (qui provoquerait la sortie immédiate du script dès qu'une commande retourne un code non nul). La capture du code de retour utilise la syntaxe :

```
1 $COMPILER "$file" >/dev/null 2>&1; actual_ret=$?
```

11.3 Tokens CHARACTER et séquences d'échappement

Le token `CHARACTER` stocke le littéral sous forme de chaîne (type `<str>` dans l'union Bison) pour pouvoir gérer les séquences d'échappement (`'\n'`, `'\t'`, `'\0'`, etc.) lors de la génération de code.

11.4 Variables locales vs paramètres

Les paramètres formels sont insérés dans `l_vars` *en tête* lors du remplissage de la table des symboles. À la génération, ils sont parcourus dans l'ordre de la liste (tête en premier), qui correspond à l'ordre inverse de déclaration. Le code de sauvegarde des registres AMD64 tient compte de cet ordre pour associer correctement chaque registre (`rdi`, `rsi`, ...) à son offset.

12 Résultats et Performances

12.1 Bilan fonctionnel

Fonctionnalité	Statut
Analyse lexicale complète	Implémenté
Analyse syntaxique (grammaire TPC)	Implémenté
Construction de l'ASA	Implémenté
Tables des symboles	Implémenté
Analyse sémantique	Implémenté
Génération NASM (types int/char)	Implémenté
Génération NASM (structures)	Implémenté
Fonctions d'E/S (syscall)	Implémenté
Gestion des avertissements	Implémenté
Tests 30/30	100%

TABLE 13 – État d'avancement du projet

12.2 Limitations connues

- Les tableaux sont reconnus syntaxiquement mais l'accès indicé (`a[i]`) n'est pas entièrement géré en génération de code.
- Les fonctions à plus de 6 paramètres ne sont pas supportées (limitation de la convention AMD64 sans passage par la pile).
- Pas de vérification des débordements arithmétiques.

13 Conclusion

Ce projet a permis de réaliser un compilateur complet pour le langage TPC, couvrant toutes les étapes de la chaîne de compilation : analyse lexicale, analyse syntaxique, construction de l'ASA, analyse sémantique et génération de code assembleur NASM.

Les 30 tests automatiques passent tous avec succès (score 100%), validant le bon fonctionnement de l'ensemble. La génération de code respecte la convention AMD64 et produit des exécutables ELF64 fonctionnels sur Linux.

Ce travail constitue une base solide pour des extensions futures : optimisation du code généré, support complet des tableaux, ou encore gestion d'appels de fonctions avec plus de 6 arguments.

A Annexes

A.1 Exemple de programme TPC compilé

```

1 int fact(int n) {
2     if (n <= 1) return 1;
3     return n * fact(n - 1);
4 }
5
6 int main(void) {
7     putint(fact(5));

```

```

8     putchar( '\n' );
9     return 0;
10 }

```

Listing 10 – Calcul de la factorielle (test/good/11_recursion.tpc)

A.2 Assembleur NASM généré (extrait)

```

1 fact:
2     push rbp
3     mov  rbp, rsp
4     sub  rsp, 8
5     mov  [rbp-8], rdi      ; parametre n
6
7     ; if (n <= 1)
8     mov  eax, [rbp-8]
9     push rax
10    mov  eax, 1
11    pop  rcx
12    cmp  ecx, eax
13    setle al
14    movzx eax, al
15    test eax, eax
16    jz   _L1_end
17    mov  eax, 1            ; return 1
18    leave
19    ret
20 _L1_end:
21     ; return n * fact(n-1)
22     mov  eax, [rbp-8]
23     push rax
24     mov  eax, [rbp-8]
25     push rax
26     mov  eax, 1
27     pop  rcx
28     sub  ecx, eax
29     mov  eax, ecx
30     mov  rdi, rax
31     call fact
32     pop  rcx
33     imul ecx, eax
34     mov  eax, ecx
35     leave
36     ret

```

Listing 11 – Code NASM pour fact(5)

A.3 Compilation et utilisation

```

# Compiler le projet
make

```

```
# Compiler un programme TPC
./bin/tpcc mon_prog.tpc          # produit mon_prog.asm

# Assembler et exécuter
nasm -f elf64 mon_prog.asm -o mon_prog.o
ld -o mon_prog mon_prog.o
./mon_prog

# Lancer tous les tests
./run_tests.sh
```